

A Real Time Application Oriented Project Report on **AGRI ASSIST**

Submitted to partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

DEEPA SRI VADLA

[23911A0514]

Under the Guidance of

Ms. G. SUREKHA

Assistant Professor



Department of Computer Science and Engineering

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

(Approved by AICTE, accredited by NAAC, NBA C permanently Affiliated to JNTUH, Hyderabad)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

2024-2025



(Approved by AICTE, accredited by NAAC, NBA C permanently Affiliated to JNTUH, Hyderabad)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

DEPARTMENT OF COMPUTER SCINENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled “**AGRI ASSIST**” is being submitted by **Deepa Sri Vadla (23911A0514)**, in partial fulfilment for the award of the degree of Bachelor of Technology in **Computer Science and Engineering**, Is a record of bonafide work carried out by her under my guidance and supervision. These results embodied in this project report have not been submitted to any other University or Institute for the award of any degree.

Internal Guide

Ms. G. Surekha
Assistant Professor

Head of Department

Dr. D. Aruna Kumari
Professor

DECLARATION

I am **Deepa Sri Vadla (23911A0514)**, Hereby declare that the project entitled, “**AGRI ASSIST**” submitted for the degree of Bachelor of Technology in Computer Science and Engineering is original and has been done by me and this work is not copied and submitted anywhere for the award of any degree.

DEEPA SRI VADLA

[23911A0514]

ACKNOWLEDGEMENT

I wish to express our sincere gratitude to the project guide, **Ms. G. Surekha**, Assistant Professor, Vidya Jyothi Institute of Technology, Hyderabad for her timely cooperation and valuable suggestions while carrying out this work. It is her kindness that made us learn more from her

I am grateful to **Dr. D Aruna Kumari**, Professor, and HOD, department of CSE, for her help and support during our academic year

I am wholeheartedly convey our gratitude to Principal **Dr. A. Srujana**, for providing the necessary infrastructure to complete this project.

I would thank my parents and all the faculty members who have contributed to our progress through the course to come to this stage.

-

DEEPA SRI VADLA

[23911A0514]

ABSTRACT

AGRI ASSIST

Agri Assist is a web-based application developed to assist farmers by providing essential agricultural insights, improving their decision-making process. The application includes a secure user authentication system to ensure data privacy and personalization. Key features include real-time weather updates, soil analysis for optimized crop selection, market price tracking for various crops, and an AI-powered crop recommendation system. The application supports multiple languages, including English, Hindi, Tamil, Telugu, and Malayalam, making it accessible to a broader audience. With a user-friendly interface and responsive design, Agri Assist ensures seamless navigation and usability. By leveraging AI and cloud computing, the platform enhances agricultural productivity and reduces uncertainties associated with farming decisions.

INDEX

S.NO	TITLE	PAGE NO.
1	INTRODUCTION	1
2	LITERATURE SURVEY	2 - 3
	2.1 KISAN SUVIDHA	2
	2.2 PUSA KRISHI	2
	2.3 CROP INSURANCE	3
	2.4 AGRI MARKET	3
	2.5 MKISAN	3
3	FEASIBILITY STUDY	4
	3.1 TECHNICAL FEASIBILITY	4
	3.2 ECONOMIC FEASIBILITY	4
	3.3 OPERATIONAL FEASIBILITY	4
4	SYSTEM REQUIREMENTS	5 - 6
	4.1 EXISTING SYSTEM	5
	4.2 PROPOSED SYSTEM	5
	4.3 HARDWARE REQUIREMENTS	6
	4.4 SOFTWARE REQUIREMENTS	6
5	SYSTEM DESIGN	7
	5.1 DESIGN OBJECTIVES	7
	5.2 DATA FLOW DIAGRAM (DFD)	7

	5.2.1 DATA FLOW DIAGRAM	8
	5.3 SYSTEM ARCHITECTURE	9
6	SYSTEM IMPLEMENTAION	10- 12
	6.1 LOGIN/SIGNUP MODULE	10
	6.2 DASHBOARD	10
	6.3 WEATHER FORECASTING	10
	6.4 CHATBOT	10
	6.5 CROP RECOMMENDATION	11
	6.6 MARKET PRICE INFORMATION	11
	6.7 GOOGLE TRANSLATE INTEGRATION	11
	6.8 Technologies Used	11
	6.8.1 HTML, CSS and Bootstrap	11
	6.8.2 Javascript	12
	6.8.3 python	12
	6.8.4 OpenWeatherMap API	12
	6.8.5 Google Translate API	12
	6.8.6 Dialog / Custom Flask Chatbot	12
7	SAMPLE CODE	13- 17
	7.1 LOGIN.HTML	13
8	SYSTEM TESTING	18- 19
	8.1 UNIT TESTING	18
	8.2 INTEGRATION TESTING	18

	8.3 USER ACCEPTANCE TESTING (UAT)	19
	8.4 OUTPUT TESTING	19
9	RESULTS & OUTPUT	20-28
	9.1 LOGIN PAGE	20
	9.2 REGISTER PAGE	21
	9.3 DASH BOARD	22
	9.4 WEATHER INFORMATION	23
	9.5 SOIL ANALYSIS	24
	9.6 MARKET PRICES	25
	9.7 CROP RECOMMENDATION	26
	9.8 CHAT BOT	27
	9.9 GOOGLE TRANSLATE	28
10	CONCLUSION	29
11	FUTURE SCOPE	30
	11.1 INTEGRATION WITH GOVERNMENT APIS	30
	11.2 IOT-BASED SOIL AND WEATHER SENSORS	30
	11.3 OFFLINE ANDROID APP	30
	11.4 SMART ALERTS	30
	11.5 REGIONAL NLP-BASED CHATBOT	30
12	REFERENCES	31

CHAPTER 1

INTRODUCTION

In recent years, agriculture has witnessed a digital revolution aimed at addressing the longstanding challenges faced by farmers. From erratic weather patterns and unpredictable market prices to the lack of real-time information and crop planning guidance, traditional farming methods are increasingly falling short. Recognizing this, the Agri Assist application was conceptualized as a modern, integrated, and user-friendly solution to empower farmers through technology.

Agri Assist is a comprehensive agricultural assistant web application that consolidates essential tools such as weather forecasting, soil analysis, market price updates, and crop recommendations into one platform. The application is designed for ease of use, offering accessibility in local languages and intuitive navigation, even for users with limited technical proficiency.

By digitizing the agricultural support system, Agri Assist aims to increase productivity, enhance decision-making, and ultimately improve farmers' livelihoods. It leverages open data sources, real-time APIs, and user feedback to provide actionable insights, bridging the gap between modern agricultural science and rural farming practices.

CHAPTER 2

LITERATURE SURVEY

Several studies and reports have highlighted the need for digitized tools in the agricultural domain. Traditional farming practices, though rooted in experience, often lack the support of dynamic data and predictive analytics. With climate change and market volatility on the rise, reliance solely on experience is becoming increasingly risky.

Existing applications such as Kisan Suvidha, Agmarknet, and Mandi Bhav offer fragmented solutions. However, none integrate weather, soil analysis, market data, and crop recommendation into a unified interface as Agri Assist does.

2.1 Kisan Suvidha

Developed by the Ministry of Agriculture and Farmers Welfare, Kisan Suvidha provides farmers with information related to weather, crop advisory, market prices, and government schemes. It aims to improve farmers' productivity and decision-making by offering timely, accurate information.

2.2 Pusa Krishi

Pusa Krishi is an app developed by the Indian Agricultural Research Institute (IARI). It offers information on various farming techniques, modern agricultural practices, and expert advice on pest management, seed selection, and crop rotation. The app also includes details about research breakthroughs and innovations in agriculture.

2.3 Crop Insurance

Crop insurance schemes like the Pradhan Mantri Fasal Bima Yojana (PMFBY) provide financial protection to farmers against crop loss due to natural calamities, pests, and diseases. These programs are vital in mitigating risks and helping farmers recover losses, ensuring their economic stability.

2.4 Agri Market

Agri Market platforms are digital initiatives aimed at improving market access for farmers. They provide information about real-time market prices, connect farmers directly with buyers, and help reduce the exploitation of middlemen. Platforms like eNAM (National Agriculture Market) are part of this effort, promoting transparency and better prices for farmers.

2.5 mKisan

mKisan is a mobile-based platform launched by the Government of India to provide farmers with essential agricultural information. It offers personalized SMS services on crop management, weather forecasts, market prices, and government schemes. The app aims to empower farmers with timely and reliable information to improve their agricultural practices.

The application takes cues from these tools but aims to overcome their limitations by offering:

- A single, integrated dashboard.
- User authentication and personalization.
- Modular navigation.
- Smart soil and crop advisory based on data rules.

CHAPTER 3

FEASIBILITY STUDY

3.1 Technical Feasibility

Agri Assist is built using widely adopted frontend technologies like HTML, CSS, and JavaScript, ensuring browser compatibility and responsive design. It interacts with APIs such as OpenWeatherMap for weather data and uses local logic for soil and crop advisory, making it technically feasible even without a constant internet connection.

3.2 Economic Feasibility

The application depends on open-source libraries and freely available public APIs, which keeps development and maintenance costs low. It is deployable on affordable cloud services or as a static website for rural connectivity zones.

3.3 Operational Feasibility

The system features a simple interface with clickable cards and dropdowns, making it suitable for users with minimal digital experience. It supports multilingual capability via Google Translate and integrates voice and chatbot support for future updates.

Agri Assist is technically sound, cost-effective, and operationally robust for immediate and widespread implementation.

CHAPTER 4

SYSTEM REQUIRMENTS

4.1 Existing System:

The development of AgriAssist requires a robust technological stack.

- The frontend is developed using React.js for the web version and Thinkable for mobile applications.
- The backend utilizes Python with Flask/Django or Node.js to manage authentication
- Data processing, and AI-driven recommendations.
- Firebase or MySQL serves as the database for storing user data and historical records.
- Essential APIs such as OpenWeather API and Soil Data API facilitate real-time data retrieval.
- The hardware requirements include a system with a minimum of 4GB RAM for development
- While end-users require a smartphone or desktop with internet connectivity.
- Future enhancements may involve integrating IoT sensors for real-time soil moisture and temperature monitoring
- Further improving agricultural decision-making.

4.2 Proposed System:

AgriAssist aims to transform traditional farming methods by providing farmers with

- real-time
- data-driven insights
- The project's scope includes delivering accurate weather forecasts, soil analysis, and crop recommendations while reducing dependency on intermediaries by offering direct access to market price trends.
- The primary objectives of AgriAssist are to develop an interactive web and mobile platform
- Integrate AI for precise agricultural analysis
- Ensure multilingual accessibility

- Facilitate efficient farm management through digital record-keeping.
- By achieving these goals, AgriAssist seeks to optimize farming practices, increase productivity, and empower farmers with innovative agricultural solutions.

4.3 Hardware Requirements:

- Device: Smartphone, tablet, or PC
- Processor: Dual-core 1.5 GHz or higher
- RAM: 2 GB minimum
- Internet: Basic internet access recommended for live data fetching

4.4 Software Requirements:

- Frontend: HTML5, CSS3, JavaScript (ES6+)
- Backend: Optional (for future enhancement like database integration)
- API Usage: OpenWeatherMap API for weather, static JSON for soil/crop
- Browser Support: Chrome, Firefox, Safari, Edge

CHAPTER 5

SYSTEM DESIGN

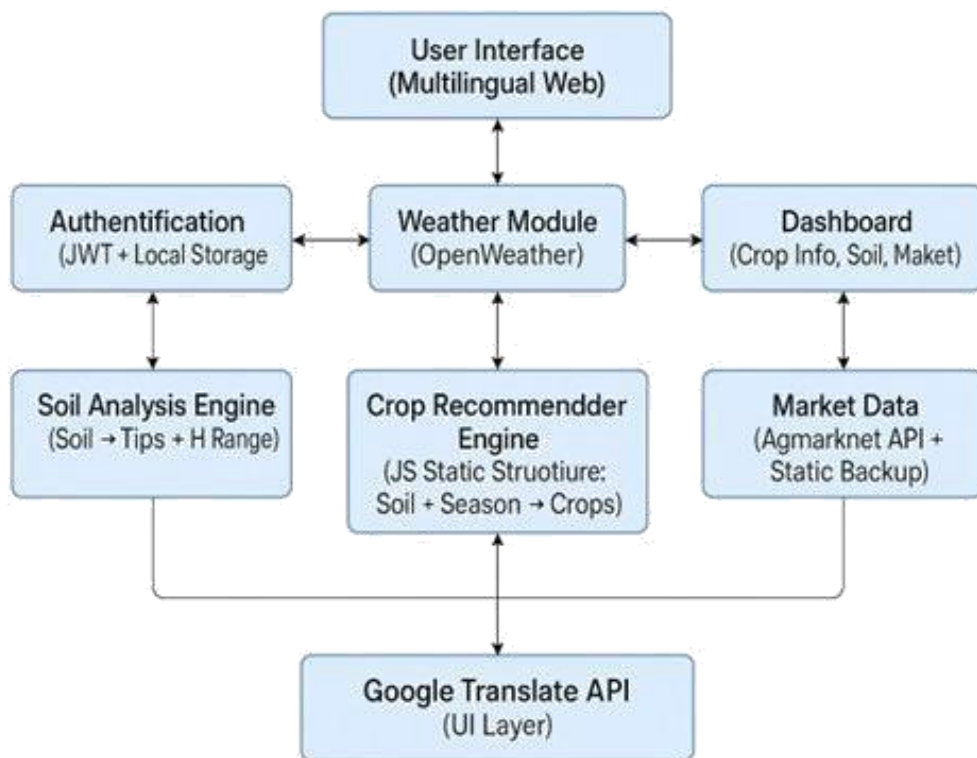
System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. For the Agri Assist application, the system is designed with a modular and scalable approach to ensure flexibility, ease of development, and future enhancements.

5.1 Design Objectives:

- Ensure a user-friendly and interactive interface.
- Provide accurate and personalized calorie, diet, and exercise recommendations.
- Support seamless API integration for nutritional data. Maintain secure and reliable data storage.
- Facilitate future integration with wearable devices and AI bots.

5.2 Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** is a graphical representation used to visualize the flow of data through a system. It shows how data is input, processed, stored, and output by the system. DFDs help stakeholders understand how systems operate and how different components interact without needing to understand the technical details of the software or hardware.



5.2.1 Data Flow Diagram

This DFD represents an **agricultural crop recommendation system**, and includes:

- **User Interface:** A multilingual web interface where users interact.
- **Authentication:** Uses JWT and local storage for secure access.
- **Soil Analysis Engine:** Analyzes soil and provides tips and helpful data (like pH range).
- **Weather Module:** Uses OpenWeather to fetch weather conditions.
- **Crop Recommender Engine:** Based on soil and season, it suggests crops using static JS logic.
- **Market Data:** Pulls data from Agmarknet API and a static backup for pricing and demand.
- **Dashboard:** Displays crop info, soil data, and market trends.
- **Google Translate API:** Adds multilingual support at the UI level.

5.3 System Architecture

System architecture is the conceptual model that defines the structure, behavior, and more views of a system. It provides a high-level blueprint of how components interact, including hardware, software, networks, and users.

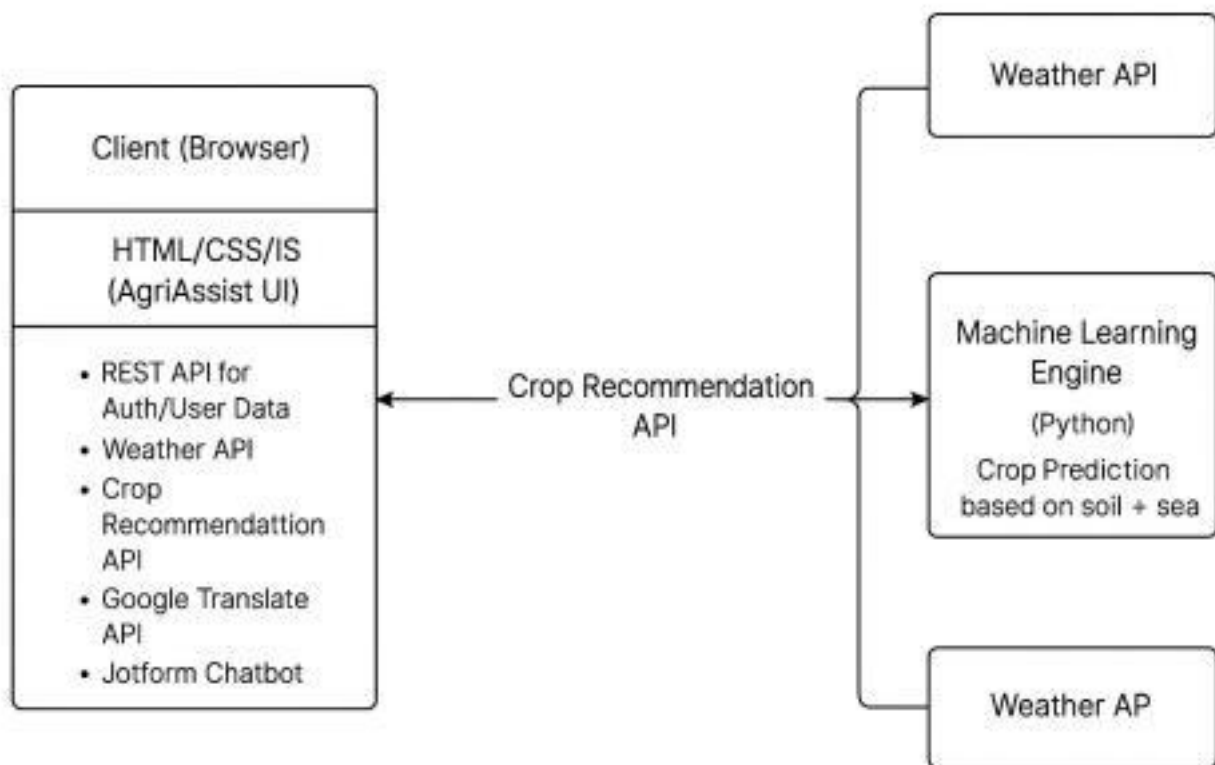


Fig 5.3 System Architecture

The AgriAssist system architecture showcases a lightweight, client-driven agricultural web application. It eliminates the need for a traditional backend by directly connecting the frontend with external services like the **Crop Recommendation API** and **Weather APIs**. This simplified, API-centric model enables fast deployment, real-time data fetching, and scalable integration with machine learning engines for intelligent crop prediction based on user inputs such as **soil type** and **season**.

CHAPTER 6

SYSTEM IMPLEMENTATION

Each feature in AgriAssist was implemented with modular coding practices, allowing easy maintenance and future upgrades. Below is a breakdown of how major features were implemented:

6.1 Login/Signup Module

- Users are authenticated via Flask routes.
- Passwords are hashed using `werkzeug.security`.
- On successful login, user sessions are maintained using Flask-Session or JWT (JSON Web Tokens).
- Signup collects user details such as name, mobile number, preferred language, and location, and stores them in the database.

6.2 Dashboard

- Serves as the control panel for the user.
- Uses Bootstrap Cards to display weather, crop suggestions, market prices, and chatbot access.
- API calls are triggered on load to fetch the latest weather and pricing data.
- Clean and intuitive layout ensures ease of navigation for rural users.

6.3 Weather Forecasting

- City or location name is passed from the frontend.
- Flask route fetches weather data from OpenWeatherMap and returns a JSON object.
- JavaScript parses this and updates the UI dynamically.

6.4 Chatbot

- Dialogflow or Flask-based logic was integrated.
- A simple keyword mapping model answers questions like:
- “What crop to grow now?”
- “What is today’s price of rice?”

- “What is the weather forecast?”
- The chatbot supports text-based interaction with optional language translation for response.

6.5 Crop Recommendation

- Based on user input like soil pH, temperature, and soil type.
- Logic coded in Python using conditionals or a trained ML model.
- Matches environmental parameters with suitable crops from a predefined database.

6.6 Market Price Information

- Admin updates crop prices daily.
- Flask fetches and returns current prices to the frontend.
- Displayed in a clean tabular format using Bootstrap Table.
- Option to filter by crop name or location.

6.7 Google Translate Integration

- JavaScript calls the Google Translate API on button click.
- Translates the entire web page dynamically without reloading.
- Improves accessibility for farmers preferring local languages.

6.8 Technologies Used

AgriAssist has been developed using a combination of frontend, backend, and external API technologies to ensure a robust, responsive, and interactive platform. Each technology was selected based on performance, ease of integration, and scalability for future enhancements.

6.8.1 HTML, CSS, and Bootstrap

- HTML5 structures the web pages of AgriAssist, ensuring semantic clarity and ease of navigation.
- CSS3 is used to style the interface, applying consistent visual aesthetics across pages.
- Bootstrap 4/5 provides responsive design capabilities, ensuring that the application adapts to different devices like mobile phones, tablets, and desktops. Grid systems and components like cards, navbars, and modals were utilized to streamline UI development.

6.8.2 JavaScript

- Enhances interactivity and responsiveness on the client side.
- Handles user inputs dynamically (e.g., loading weather data without full page reload).
- Integrates with external APIs (like Google Translate) through AJAX and fetch requests.

6.8.3 Python (Flask Framework)

- Serves as the core backend engine for routing, logic handling, and data exchange.
- Flask is lightweight, modular, and well-suited for building REST APIs and integrating machine learning models.
- Routes are defined to handle operations like user login, signup, fetching weather data, crop recommendations, and database interactions.

6.8.4 OpenWeatherMap API

- Fetches real-time weather information such as temperature, humidity, rainfall chances, and wind speed based on user location.
- Integrated with the Flask backend via HTTP GET requests.
- Provides accurate and location-specific forecasts crucial for crop planning and irrigation decisions.

6.8.5 Google Translate API

- Enables multilingual support within the platform by translating page content into the user's preferred language (e.g., Telugu, Hindi, English).
- Ensures accessibility for farmers who may not be fluent in English or Hindi.
- Integrated via JavaScript to dynamically render translated content on button click.

6.8.6 Dialogflow / Custom Flask Chatbot

- Used to implement a smart chatbot capable of responding to user queries related to crop suggestions, market prices, weather, and more.
- In some cases, a Flask-based logic engine was built with keyword mapping to simulate natural interactions.
- Offers contextual help in simple, farmer-friendly language.

CHAPTER 7

SAMPLE CODE

7.1 Login.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>AgriAssist Login</title>

  <style>

    .auth-container {

      max-width: 400px;

      margin: auto;

      padding: 20px;

      font-family: Arial, sans-serif;

    }

    .form-container {

      display: none;
```

```
}
```

```
.form-container.active {  
    display: block;  
}
```

```
.form-group {  
    margin-bottom: 15px;  
}
```

```
label {  
    display: block;  
    margin-bottom: 5px;  
}
```

```
input {  
    width: 100%;  
    padding: 8px;  
box-sizing: border-box;  
}
```

```
.error-message {  
  
    color: red;  
  
    font-size: 0.9em;  
  
}
```

```
.switch-form {  
  
    margin-top: 10px;  
  
}
```

```
.switch-form button {  
  
    background: none;  
  
    border: none;  
  
    color: blue;  
  
    cursor: pointer;  
  
    text-decoration: underline;  
  
    padding: 0;  
  
}
```

```
button[type="submit"] {  
  
width: 100%;
```

```
padding: 10px;

margin-top: 10px;

background-color: green;

color: white;

border: none;

cursor: pointer;

}
```

```
button[type="submit"]:hover {

    background-color: darkgreen;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div id="auth-section" class="auth-container">
```

```
<!-- Login Form -->
```

```
<div id="login-form" class="form-container
```

```
active"> <h2>Login to AgriAssist</h2>
```

```
<form id="loginForm">
```

```
<div class="form-group">
```



```
<label for="login-email">Email:</label>

<input type="email" id="login-email" name="email" required>

<div id="login-email-error" class="error-message"></div>

</div>

<div class="form-group">

  <label for="login-password">Password:</label>

  <input type="password" id="login-password" name="password" required>

  <div id="login-password-error" class="error-message"></div>

</div>

<button type="submit">Login</button>

</button>

</div>

</form>

</div>

</script>

</body>

</html>
```

CHAPTER 8

SYSTEM TESTING

Testing ensures that **Agri Assist** is robust, secure, reliable, and user-friendly. Multiple levels of testing were conducted throughout the development life cycle to verify the correctness of each module, ensure proper integration of components, and validate the system from an end-user perspective.

8.1 Unit Testing

Individual modules were tested in isolation to validate their functionality and correctness.

- **Login/Signup:** Verified password encryption using hashing, ensured proper validation for input fields like name, mobile number, and password.
- **Crop Recommendation:** Tested the logic against a variety of soil pH levels, temperatures, and soil types to ensure appropriate crop suggestions.
- **Weather API:** Checked responses for both valid and invalid city names. Also tested for error handling in case of API failures or connectivity issues.
- **Market Prices:** Verified correct retrieval of crop price data from the database and validated display in the UI.
- **Chatbot:** Checked keyword-response mappings and fallback message handling when unknown queries are entered.
- **Google Translate:** Ensured the page translation script executes properly and updates content without breaking page layout or functionality.

8.2 Integration Testing

Focus was placed on how different modules work together in a unified environment. Integration testing confirmed that data flows between modules as expected and that combined functionalities work seamlessly.

- **Dashboard Integration:** Verified that the dashboard loads all widgets (weather, crop recommendation, market prices) together without delay or visual overlap.
- **Signup → Language Preference → Dashboard:** Ensured that selected language during signup is applied automatically on the dashboard after login.
- **Chatbot and Crop Logic:** Confirmed that the chatbot correctly triggers the crop recommendation engine and provides meaningful suggestions.
- **Weather Module and Dashboard:** Tested dynamic weather data fetching via API and confirmed real-time update on the user interface.
- **Google Translate and UI:** Checked that changing the interface language using Google Translate does not affect the layout, navigation, or data presentation.

8.3 User Acceptance Testing (UAT)

- User Acceptance Testing was performed to assess the system's usability and effectiveness from an end-user's perspective.
- Real users—including farmers, agricultural students, and faculty—interacted with the platform on both mobile and desktop devices.
- Feedback was gathered on interface clarity, language accessibility, and overall usefulness of features.
- Based on the insights, improvements were made to enhance user experience.

8.4 Output Testing

- Output Testing verifies that the system produces correct, expected, and meaningful results when provided with input data.
- In AgriAssist, this included checking crop recommendations based on soil and weather inputs, validating API responses from the weather module, and ensuring market prices and chatbot replies were accurate.
- This testing confirmed that all functional outputs aligned with user expectations and application requirements.

CHAPTER 9

RESULTS & OUTPUT

9.1 LOGIN PAGE

The login interface provides secure access to AgriAssist's features, allowing users to authenticate using their email and password.

A clean and user-friendly design ensures ease of use, with a prompt to switch to the Sign-Up form if the user doesn't have an account.

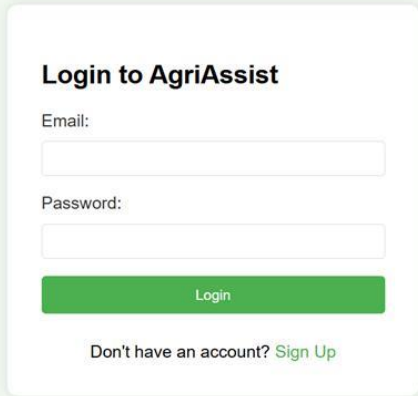
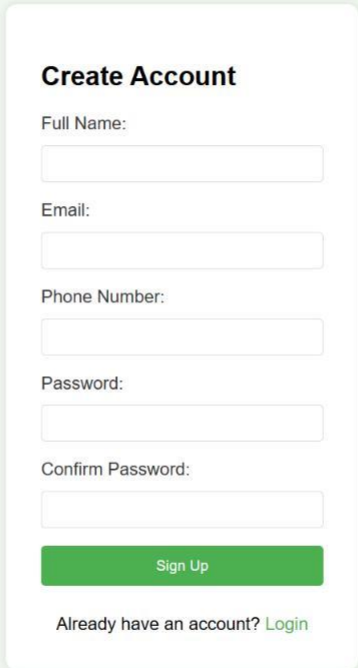
The image shows a login form titled "Login to AgriAssist" centered on a light green background. The form is a white rounded rectangle containing the following elements: the title "Login to AgriAssist", an "Email:" label followed by a text input field, a "Password:" label followed by a text input field, a green "Login" button, and a link "Don't have an account? Sign Up" where "Sign Up" is in green text.

Fig 9.1 Login page

9.2 REGISTER PAGE

The sign-up form allows new users to register by providing essential details such as name, email, phone number, and password.

It ensures secure account creation with password confirmation and a clear link to redirect existing users to the login page



The image shows a registration form titled "Create Account" centered on a light green background. The form is a white card with rounded corners. It contains the following elements from top to bottom: a title "Create Account", a label "Full Name:" followed by a text input field, a label "Email:" followed by a text input field, a label "Phone Number:" followed by a text input field, a label "Password:" followed by a text input field, a label "Confirm Password:" followed by a text input field, a green "Sign Up" button, and a link "Already have an account? Login" where "Login" is in green text.

Fig 9.2 Registration Page

9.3 DASH BOARD

The AgriAssist dashboard offers quick access to key modules such as weather updates, soil analysis, market prices, and crop recommendations.

Designed for ease of navigation, it empowers users to make informed farming decisions based on real-time data and analysis.

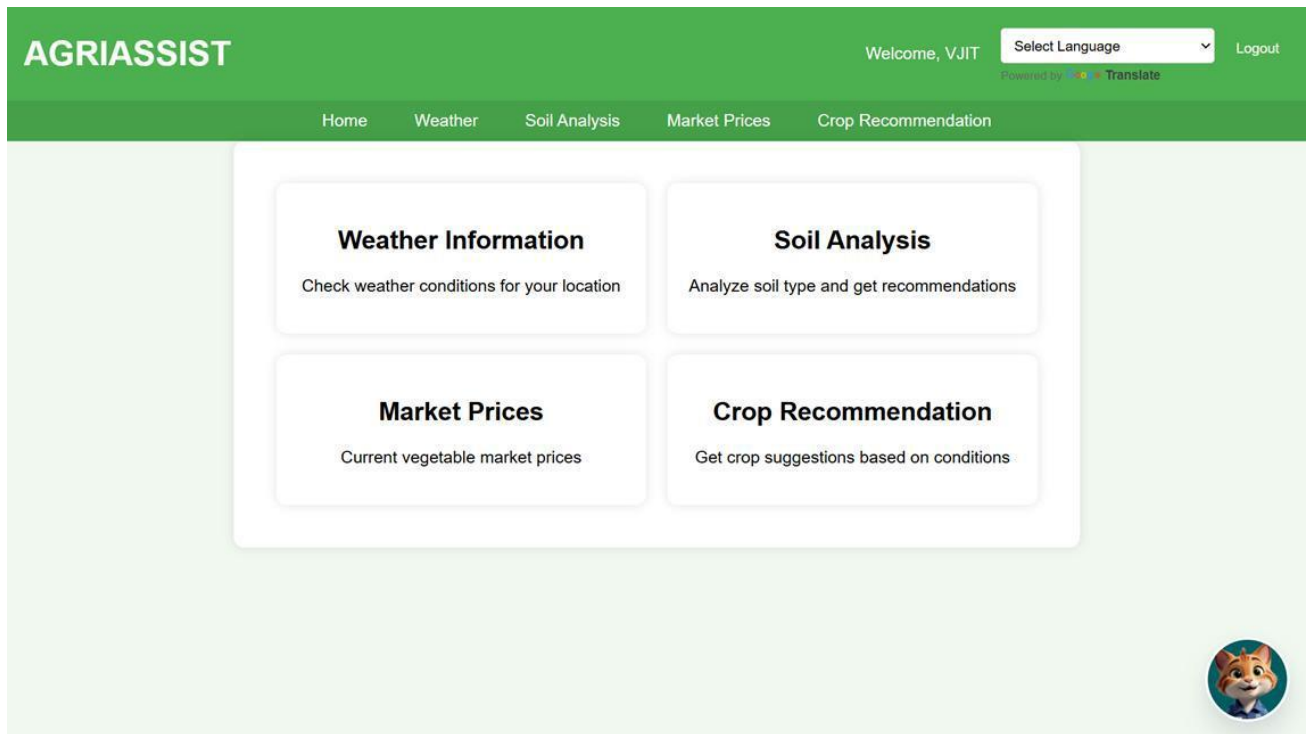


Fig 9.3 Dash Board

9.4 WEATHER INFORMATION

The Weather Information module allows users to input a city name and retrieve real-time weather data including temperature, humidity, and sky conditions.

This feature helps farmers make climate-aware decisions for irrigation, planting, and harvesting.

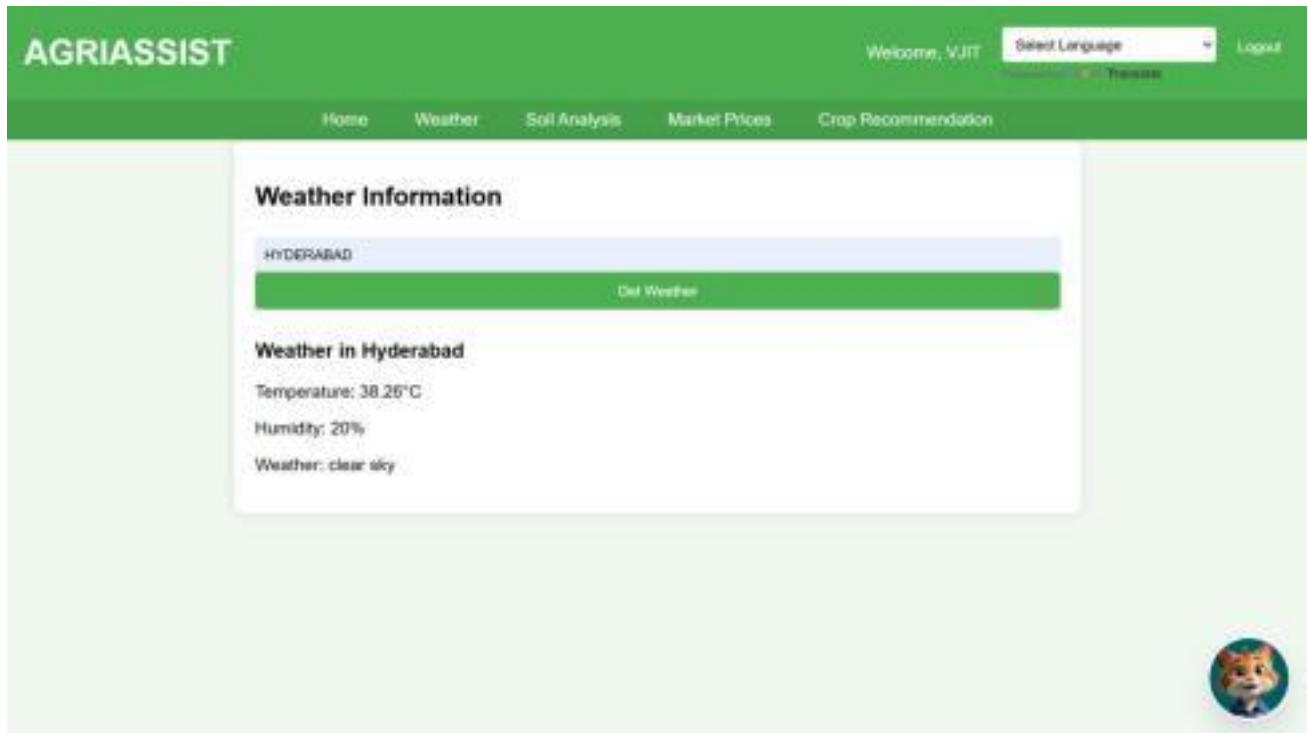
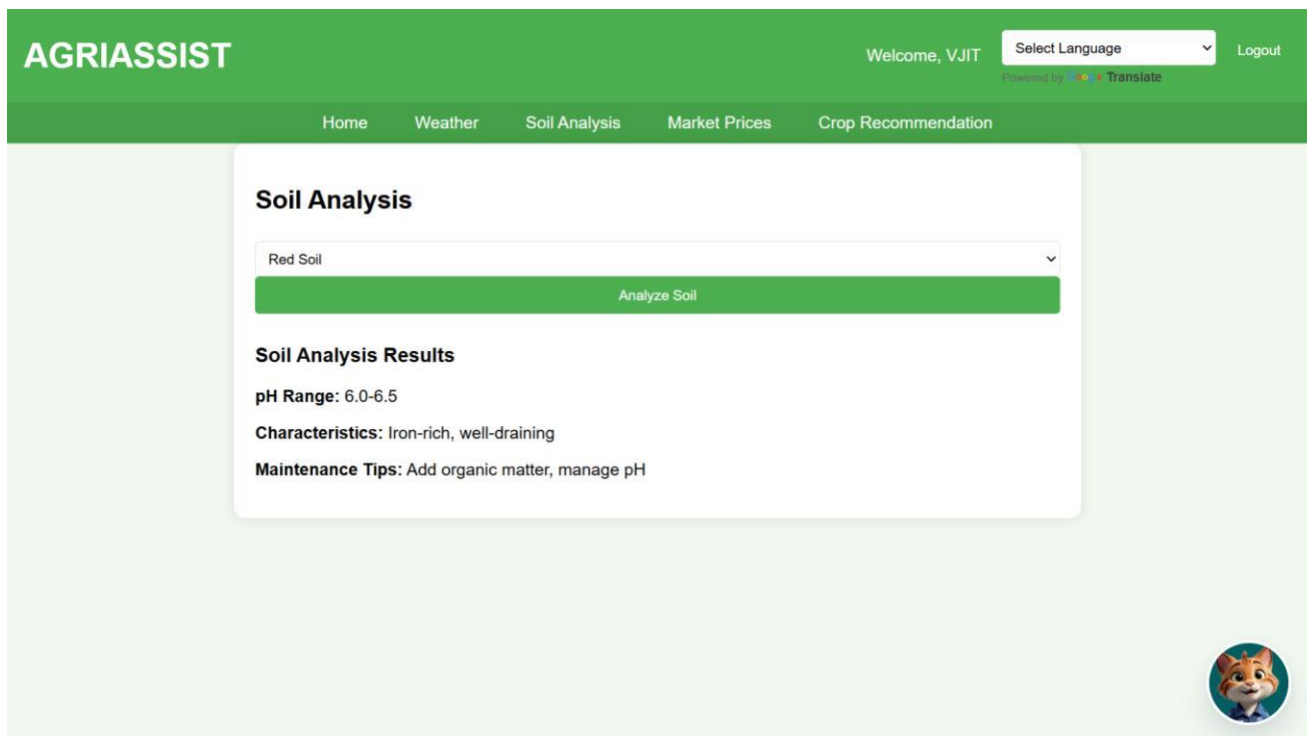


Fig 9.4 Weather Information

9.5 SOIL ANALYSIS

- **Soil Type Selection:** The user selects a soil type from a dropdown menu to initiate soil analysis based on their agricultural field conditions.
- **Soil Analysis Result:** Upon selection, the application provides key details such as pH range, soil characteristics, and maintenance tips to guide effective soil management.



The screenshot displays the AGRIASSIST web application interface. The top navigation bar is green with the logo 'AGRIASSIST' on the left, 'Welcome, VJIT' in the center, and a 'Select Language' dropdown menu on the right. Below the navigation bar is a horizontal menu with links: 'Home', 'Weather', 'Soil Analysis', 'Market Prices', and 'Crop Recommendation'. The main content area features a white card titled 'Soil Analysis'. Inside the card, there is a dropdown menu currently showing 'Red Soil'. Below the dropdown is a green button labeled 'Analyze Soil'. Underneath the button, the 'Soil Analysis Results' are displayed: 'pH Range: 6.0-6.5', 'Characteristics: Iron-rich, well-draining', and 'Maintenance Tips: Add organic matter, manage pH'. In the bottom right corner of the page, there is a small circular avatar of a cartoon fox.

Fig 9.5 Soil Analysis

9.6 MARKET PRICES

The Market Prices section redirects users to an external government portal that provides daily market-wise and commodity-wise price reports.

Users can select their state and district to retrieve updated agricultural commodity prices relevant to their region.

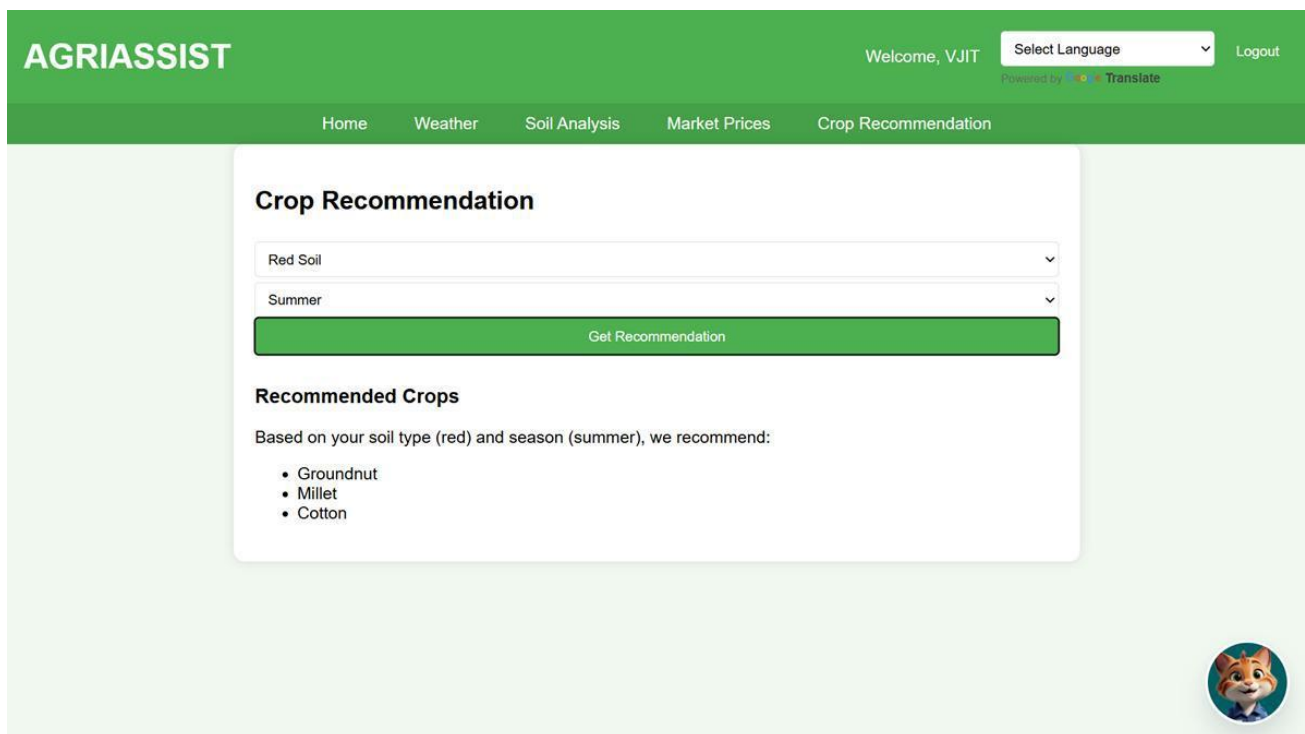
<u>Market-wise, Commodity-wise Daily Report- District-wise</u>	
State:	--Select-- ▾
District :	--Select-- ▾
Submit	

Fig 9.6 Market Prices

9.7 CROP RECOMMENDATION

The **Crop Recommendation** module suggests optimal crops based on user-selected soil type and season.

It leverages predefined data mappings to display suitable crops such as Groundnut, Millet, and Cotton for red soil in summer.



The screenshot displays the AGRIASSIST web application interface. The top navigation bar is green with the AGRIASSIST logo on the left, a welcome message 'Welcome, VJIT' in the center, and a language selection dropdown and 'Logout' link on the right. Below this is a secondary green navigation bar with links for Home, Weather, Soil Analysis, Market Prices, and Crop Recommendation. The main content area features a white card titled 'Crop Recommendation'. Inside the card, there are two dropdown menus: the first is set to 'Red Soil' and the second to 'Summer'. Below these is a green button labeled 'Get Recommendation'. Underneath the button, the section 'Recommended Crops' is displayed, followed by the text 'Based on your soil type (red) and season (summer), we recommend:'. A bulleted list then shows the recommended crops: Groundnut, Millet, and Cotton. In the bottom right corner of the page, there is a small circular avatar of a cartoon fox.

Fig 9.7 Crop Recommendation

9.8 CHAT BOT

The **AGRI BOT** is an integrated AI support agent designed to assist users with agricultural queries and guidance.

Users can interact with the bot to explore agriculture tips, insights, and receive help navigating the AGRIASSIST platform.

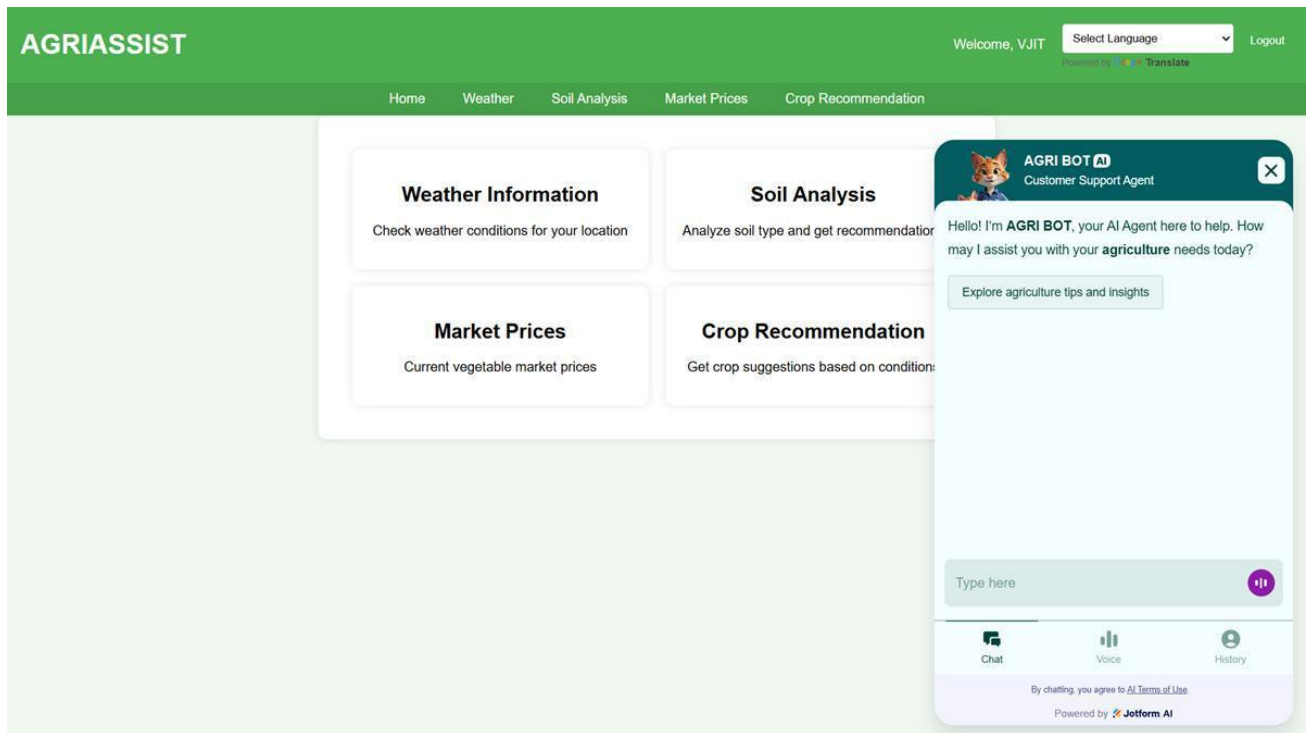


Fig 9.8 Chat Bot

9.9 GOOGLE TRANSLATE

The **Language Selection** dropdown enables users to view the AGRIASSIST platform in their preferred language.

Powered by Google Translate, it supports multiple regional and international languages, enhancing accessibility for diverse users.

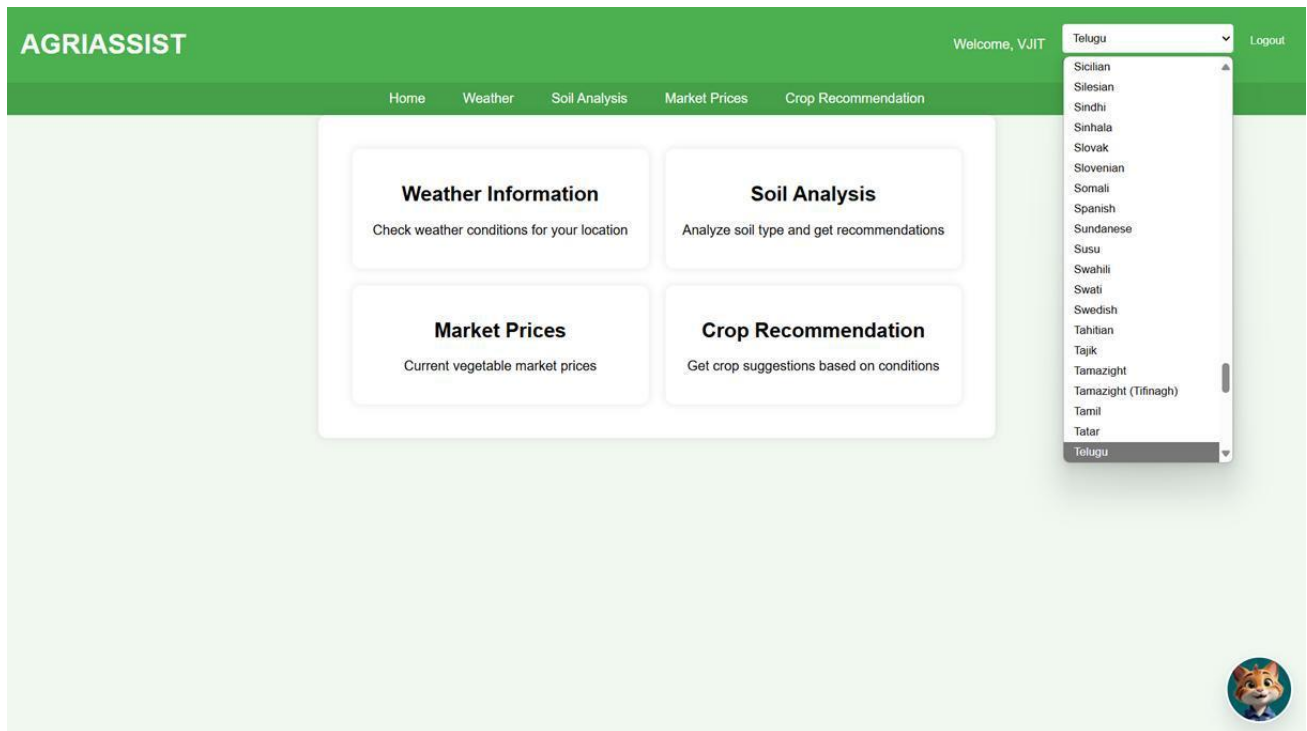


Fig 9.9 Google Translate

CHAPTER 10

CONCLUSION

AgriAssist demonstrates that a farmer-centric digital platform can significantly improve agricultural outcomes when designed with simplicity, accessibility, and reliability in mind. By integrating critical features such as crop advisory, live weather updates, and real-time market data, AgriAssist enables informed decision-making among farmers. The system's modular structure allows seamless additions in future versions, and the use of open APIs ensures up-to-date, real-world data. The Google Translate integration breaks language barriers, and the clean UI ensures usability even for those unfamiliar with smartphones or computers. AgriAssist doesn't just deliver information—it empowers farmers with clarity, context, and confidence to act.

CHAPTER 11

FUTURE SCOPE

To elevate AgriAssist from a digital assistant to a holistic agri-management tool, the following enhancements are proposed:

11.1 Integration with Government APIs

- a. Automate updates about schemes (PM-KISAN, Rythu Bandhu).
- b. Verify beneficiaries and subsidy status in real-time.

11.2 IoT-Based Soil and Weather Sensors

Collect real-time data directly from the field.

Auto-suggest crop or irrigation actions.

11.3 Offline Android App

Full offline functionality with background data sync.

Enable farmers with low connectivity to access saved insights.

11.4 Smart Alerts

Push notifications for sudden weather changes or market fluctuations.

Alerts for irrigation, fertilization, or disease risks (future).

11.5 Regional NLP-Based Chatbot

Develop a full chatbot supporting natural language queries in Telugu, Hindi, etc.

Responds contextually using AI-driven conversational models.

CHAPTER 12

REFERENCES

- [1] Mishra, P., Singh, S., & Verma, S. (2021). Crop recommendation system for precision agriculture using soil parameters. *Procedia Computer Science*, 178, 691–698.
<https://doi.org/10.1016/j.procs.2020.12.123>
- [2] Patil, A., & Birajdar, G. (2022). Smart agriculture: A crop advisory system using IoT and machine learning. *International Journal of Advanced Research in Computer Science*, 13(2), 112–117. <http://www.ijarcs.info/index.php/Ijarcs/article/view/7380>
- [3] Kumar, R., & Joshi, A. (2020). Machine learning-based crop advisory system using soil health parameters. *International Conference on Intelligent Computing and Control Systems (ICICCS)*, 1136–1141. <https://ieeexplore.ieee.org/document/9257763>
- [4] Sharma, R., & Yadav, D. (2023). Crop prediction and advisory using ensemble learning on soil datasets. *Journal of Agriculture and Food Research*, 5, 100284.
<https://doi.org/10.1016/j.jafr.2023.100284>
- [5] Jain, M., & Verma, R. (2021). Development of a web-based dashboard for real-time agricultural data monitoring. *International Journal of Computer Applications*, 175(12), 18–22. <https://www.ijcaonline.org/archives/volume175/number12/jain-2021-ijca-920147.pdf>
- [6] Gupta, R., & Saxena, N. (2020). Smart agriculture using IoT and web-based dashboards. *Proceedings of the 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 1–6. <https://ieeexplore.ieee.org/document/9203020>
- [7] Ahmed, S., & Bhatia, A. (2019). Interactive agri-dashboards for farm monitoring and crop advisory. *IEEE International Conference on Data Science and Engineering (ICDSE)*, 145–149. <https://ieeexplore.ieee.org/document/8970784>.